Timers: setTimeout, setInterval, setImmediate & the Event Loop

10 min read · Updated 24 Jul 2026 · Topic 25 of 70

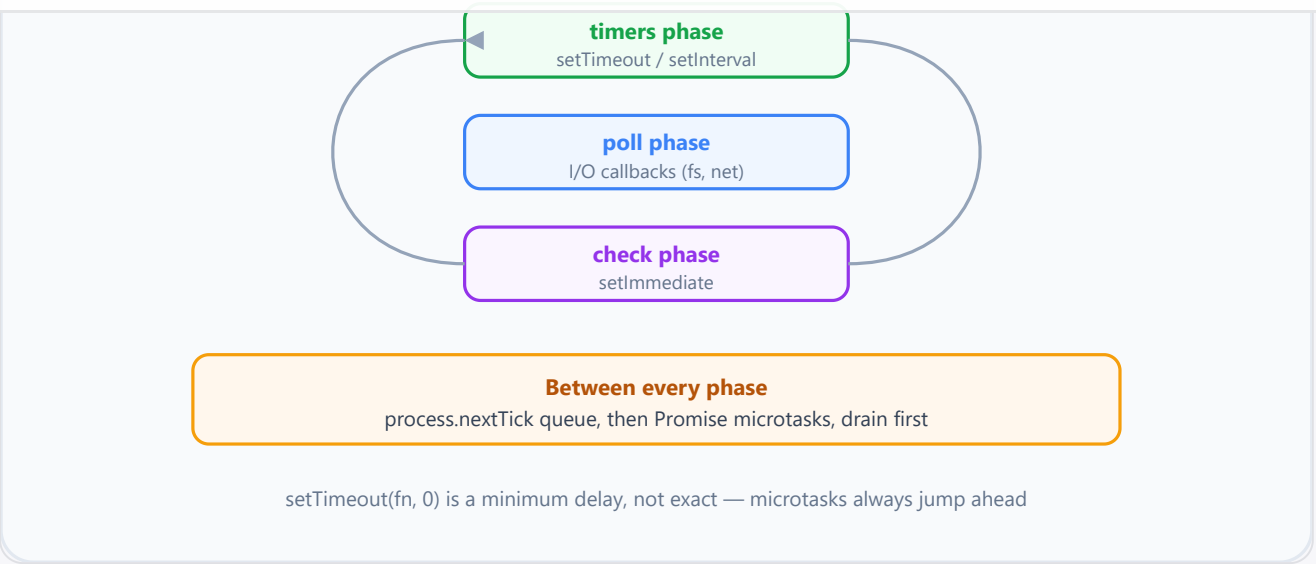
Why it matters

Timers look like the simplest API in Node — "run this later" — yet they are where the event loop's behaviour becomes visible, and where subtle ordering bugs and memory leaks hide. You use timers constantly: retry backoffs, request timeouts, polling, debouncing, rate limiting, scheduled cleanup. Understanding *when* a timer callback actually runs, and how that interacts with promises and I/O, is what lets you reason about ordering instead of guessing.

Interviewers love timers because a single question — "what order do these logs print?" — instantly reveals whether you understand the event loop, microtasks versus macrotasks, and the fact that `setTimeout(fn, 0)` does not mean "now." These are not trivia; timeout handling and leaked intervals are real production concerns. This topic ties the whole chapter's async foundation together.

The concept, simply

Node gives you three scheduling primitives. `setTimeout(fn, ms)` runs `fn` once after *at least* `ms` milliseconds. `setInterval(fn, ms)` runs `fn` repeatedly every `ms` until you stop it. `setImmediate(fn)` runs `fn` on the very next loop iteration, after the current I/O phase. Each returns a handle you cancel with `clearTimeout`, `clearInterval`, or `clearImmediate`. Two more sit alongside them at higher priority: `process.nextTick(fn)` and **Promise microtasks**, which drain *before* the loop moves on. The single most important truth: timers specify a *minimum* delay, not an exact one — the callback runs when the event loop reaches the timers phase and the thread is free, never sooner.



setTimeout fires in the timers phase, setImmediate in check. process.nextTick and Promise microtasks drain between every phase, ahead of both.

How it works (deep)

The event loop runs in **phases**, cycling continuously. The three that matter for timers are: the **timers** phase, where expired `setTimeout` / `setInterval` callbacks run; the **poll** phase, where I/O callbacks (completed file reads, incoming sockets) run and where the loop will wait for new I/O; and the **check** phase, where `setImmediate` callbacks run. Between *every* phase transition, Node drains two higher-priority queues first: the `process.nextTick` queue, then the Promise microtask queue. This is why an `await` continuation or a `.then` callback always runs before the next `setTimeout`, even a zero-delay one.

That ordering explains the classic puzzle. `setTimeout(fn, 0)` does not run immediately: the callback is registered, the current synchronous code finishes, then microtasks drain, and only when the loop reaches the timers phase does `fn` run. Meanwhile a `Promise.resolve().then( ... )` queued in the same tick runs first because microtasks jump ahead of the next macrotask. And a `process.nextTick` beats even that, because its queue drains before microtasks. The hierarchy per turn is: current synchronous code → `process.nextTick` queue → Promise microtasks → next event-loop phase (timers/poll/check).

The relationship between `setTimeout(fn, 0)` and `setImmediate(fn)` is famously order-dependent at the top level. Called from the main module, their order is non-

`setImmediate` *always* fires first, because `check` is the very next phase after `poll`, whereas `timers` is a full loop away. That deterministic difference is a common interview follow-up.

Timers also carry practical weight. A `setInterval` that is never cleared runs forever and, if its callback closes over large objects, leaks memory — a frequent cause of slowly growing servers. Timers also keep the process alive: by default a pending timer prevents Node from exiting. If you have a background poller that should not block shutdown, call `.unref()` on the handle so the loop can exit while it is the only thing left. Finally, `setInterval` does not guarantee even spacing: if the callback (or a blocked loop) takes longer than the interval, executions bunch up or drift, which is why a self-scheduling `setTimeout` is often better for reliable spacing.

A detail that bites people building long schedulers: Node stores a timer's delay as a **signed 32-bit integer**, so the maximum is `2147483647` ms (about 24.8 days). Pass anything larger — or `NaN`, `Infinity`, or a negative — and Node clamps the delay to `1` ms and emits a `TimeoutOverflowWarning`. So a naive `setTimeout(fn, days * 86400000)` for a 30-day reminder fires almost immediately instead of a month later. Long delays must be chunked (re-arm the timer in bounded steps) or driven by a real scheduler that compares wall-clock time. On the input side, a non-integer delay is truncated and a delay below `1` is treated as `1`.

The modern `node:timers/promises` module is worth reaching for by default in async code. `setTimeout(ms, value, { signal, ref })` returns a promise that resolves to `value` after the delay, so `await setTimeout(200)` replaces the hand-rolled `new Promise(r => setTimeout(r, 200))` wrapper. It accepts an `AbortSignal` so a pending delay can be cancelled (rejecting with an `AbortError`), and a `{ ref: false }` option that mirrors `.unref()`. There is also a `setInterval` async-generator form — `for await (const _ of setInterval(1000, null, { signal }))` — that yields on each tick and, because your loop body is awaited before the next yield, sidesteps the bunching problem that the callback-based `setInterval` suffers from. It is the cleanest way to write a cancellable, non-overlapping poller.

Code examples

The ordering that interviewers ask about — predict the output:

JAVASCRIPT Copy

```
console.log('1: sync start');

setTimeout(() => console.log('2: setTimeout 0'), 0);

Promise.resolve().then(() => console.log('3: promise microtask'));

process.nextTick(() => console.log('4: nextTick'));

console.log('5: sync end');

// Output order:
// 1: sync start
// 5: sync end
// 4: nextTick          (nextTick queue drains first)
// 3: promise microtask (microtasks next)
// 2: setTimeout 0      (timers phase last)
```

Clearing timers to cancel work, and using `unref` so a poller does not block exit:

JAVASCRIPT Copy

```
// A cancellable interval:
const handle = setInterval(() => console.log('poll'), 1000);
// Later, stop it — otherwise it runs forever and can leak memory:
clearInterval(handle);

// A background heartbeat that must not keep the process alive on shutdown
const beat = setInterval(() => sendHeartbeat(), 30_000);
beat.unref(); // process can exit even while this timer is pending
```

A promise-based delay and a timeout wrapper — the modern way to add deadlines:

JAVASCRIPT Copy

```
import { setTimeout as delay } from 'node:timers/promises';
```

```
// Await a delay without wrapping setTimeout by hand:
console.log('ran after ~200ms');

// Race any promise against a deadline using AbortSignal.timeout:
async function fetchWithTimeout(url, ms) {
  const res = await fetch(url, { signal: AbortSignal.timeout(ms) });
  return res.json();
}
// If the request exceeds ms, it rejects with a TimeoutError.
```

Reliable fixed spacing with self-scheduling `setTimeout` instead of `setInterval`:

JAVASCRIPT Copy

```
// setInterval can bunch up if the task runs long; this waits for completi
// THEN schedules the next run — guaranteeing a real gap between execution
async function poll(task, gapMs) {
  while (true) {
    await task();
    await new Promise((r) => setTimeout(r, gapMs));
  }
}
```

Common mistakes & gotchas

- **Believing `setTimeout(fn, 0)` runs "immediately."** It runs after the current sync code, the `nextTick` queue, and all microtasks — and only when the timers phase is reached. Zero is a minimum, not an exact delay.
- **Never clearing a `setInterval`.** It runs forever and, if it captures large state, leaks memory. Always keep the handle and `clearInterval` when done.
- **Expecting `setInterval` to space runs evenly.** If the callback or a blocked event loop overruns the interval, executions drift or pile up. Self-schedule with `setTimeout` for guaranteed gaps.
- **Blocking the event loop and blaming the timer.** A long synchronous computation delays *all* timers, because callbacks only run when the loop is free. The timer is fine; the blocking code is the problem.

an I/O callback.

- **Forgetting timers keep the process alive.** A pending timer blocks a clean exit. Use `.unref()` for background timers that should not.
- **Starving the loop with recursive `process.nextTick`.** Because `nextTick` drains fully before the loop advances, an ever-growing `nextTick` chain can starve I/O and timers entirely.
- **Scheduling a delay longer than ~24.8 days.** The delay is a signed 32-bit int; anything over `2147483647` ms (or `NaN` / `Infinity` /negative) is clamped to `1` ms with a `TimeoutOverflowWarning`, so a "fire in 30 days" timer fires now. Chunk long waits or use a wall-clock scheduler.
- **Leaking a promise-based timer's `AbortSignal` listener.** When you race `timers/promises` `setTimeout` against other work, pass an `AbortController` and abort the loser; otherwise the pending timer (and its signal listener) lingers until it fires, quietly retaining whatever the callback closed over.

Best practices

- **Always store and clear timer handles;** treat an uncleared `setInterval` as a leak.
- **Prefer self-scheduling `setTimeout` over `setInterval`** when even spacing or waiting for async work matters.
- **Use `node:timers/promises`** (`setTimeout` / `setInterval` promise forms) for clean `await`-based delays.
- **Implement deadlines with `AbortSignal.timeout`**, which cancels the underlying work rather than just ignoring a late result.
- **Call `.unref()` on background timers** that should not keep the process alive.
- **Keep event-loop turns short;** offload heavy CPU work to worker threads so timers and I/O stay responsive.
- **Reason about ordering explicitly:** sync → `nextTick` → microtasks → timers/check, every turn.

Q: Does `setTimeout(fn, 0)` run `fn` immediately? No. It schedules `fn` to run in the timers phase after the current synchronous code completes and after the `nextTick` and microtask queues drain. Zero is a minimum delay; the actual delay depends on when the loop reaches the timers phase and whether the thread is free.

Q: In what order do `setTimeout(0)`, `Promise.then`, and `process.nextTick` run? `process.nextTick` first (its queue drains before everything), then Promise microtasks, then the `setTimeout` callback in the timers phase. Microtasks always beat macrotasks; `nextTick` beats even microtasks.

Q: `setTimeout(fn,0)` vs `setImmediate(fn)` — which runs first? At the top level it is non-deterministic and depends on loop timing. But *inside an I/O callback*, `setImmediate` always runs first, because `check` is the phase immediately after `poll`, whereas the timers phase is a full loop iteration away.

Q: Why did my `setTimeout(fn, 1000)` fire much later than 1 second? Because a timer's delay is a minimum. If the event loop was busy — a long synchronous task or a backlog of callbacks — `fn` waits until the loop is free and reaches the timers phase. The timer was not late; the loop was blocked.

Q: How can a `setInterval` cause a memory leak? If it is never cleared and its callback closes over objects, those objects are retained for the life of the process, and the interval itself keeps running. Always clear intervals you no longer need, and use `.unref()` for ones that should not block shutdown.

Quick: why prefer self-scheduling `setTimeout` over `setInterval` for polling an API? `setInterval` fires on a fixed schedule regardless of whether the previous call finished, so slow responses cause overlapping or bunched requests. Re-scheduling with `setTimeout` after each call completes guarantees a real gap and no overlap.

Cheat-card

- `setTimeout` = once after ≥ ms; `setInterval` = repeat; `setImmediate` = next loop, check phase.
- Cancel with `clearTimeout` / `clearInterval` / `clearImmediate`.
- Per turn: sync → `process.nextTick` → Promise microtasks → timers/poll/check.
- Delay is a *minimum*, never exact — a blocked loop delays every timer.
- Inside an I/O callback, `setImmediate` beats `setTimeout(0)` deterministically.
- Uncleared `setInterval` = leak + keeps process alive; use `.unref()` for background timers.
- Even spacing / await async work → self-scheduling `setTimeout`, not `setInterval`.
- Modern: `node:timers/promises` for `await` delays; `AbortSignal.timeout` for deadlines.

QUICK CHECK — no marks, just for you.

1. In what order do process.nextTick, a Promise.then microtask, and a setTimeout(fn, 0) callback run when queued in the same tick?

- ○ process.nextTick first, then the Promise microtask, then the setTimeout callback in the timers phase
- ○ setTimeout first because its delay is 0, then nextTick, then the microtask
- ○ The Promise microtask first, then setTimeout, then nextTick
- ○ All three run in the exact order they were written in the source

2. Called from inside an I/O callback, which fires first: setTimeout(fn, 0) or setImmediate(fn)?

- ◯ setTimeout(fn, 0) always fires first because 0 ms is the shortest delay
- ◯ setImmediate always fires first, because the check phase runs immediately after the poll phase whereas timers is a full loop away
- ◯ The order is still non-deterministic even inside an I/O callback
- ◯ They fire simultaneously since both represent zero delay

Check answer

3. What happens if you schedule setTimeout(fn, ms) with a delay larger than 2147483647 ms (~24.8 days)?

- ◯ Node schedules it correctly using a 64-bit delay counter
- ◯ Node throws a RangeError and the timer is never scheduled
- ◯ Node clamps the delay to 1 ms and emits a TimeoutOverflowWarning, so the timer fires almost immediately instead of much later
- ◯ Node rounds the delay down to exactly 24.8 days and fires then

Check answer

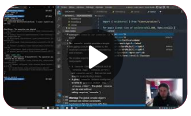
Node.js Backend Kit

LEARN MORE

ARTICLES & DOCS

- Timers | Node.js Documentation
  nodejs.org
- Discover JavaScript Timers
  nodejs.org

VIDEOS



Node.js Timers (setTimeout, setInterval, setImmediate)
Thomas Gentilhomme

Hand-picked references for this lesson.